

GROUP 2 · 70MM AI · SCRIPT INTELLIGENCE

70MM AI — Script Intelligence Engine

Complete Project Documentation

An advanced studio-grade screenplay intelligence platform powered by Google Gemini AI. Transform raw concepts into Hollywood screenplays, generate character bibles, diagnose narrative flaws with a 12-point Script Doctor, and produce studio-level coverage.

12	8	7	3
API ENDPOINTS	CORE FEATURES	PIPELINE STEPS	AI MODELS

Python 3.13 · Flask 3.1.0 · Google Gemini AI · HTML5 + Vanilla JS · Running on :5000

📋 TABLE OF CONTENTS

1. Project Overview
2. System Architecture
3. Technology Stack
4. Project File Structure
5. Step-by-Step: What Was Built (12 Steps)
6. 7-Step Screenplay Pipeline
7. User Interface
8. API Endpoints Reference
9. Backend Implementation
10. Frontend Implementation
11. Environment & Configuration
12. How to Run the Project
13. Why Google Gemini vs MuRIL / AI4Bharat
14. Model Comparison Matrix

SECTION 01

🎬 Project Overview

70MM AI — Group 2: Script Intelligence Engine is an AI-powered, studio-grade web application built as part of the 70MM AI Four-Group Cinematic Architecture. It enables professional screenwriters, script consultants, and directors to convert stories to properly formatted Hollywood screenplays, analyze characters, diagnose narrative weaknesses, and generate studio-level coverage — all powered by Google Gemini AI.

3	450	760	5
FRONTEND FILES	LINES IN APP.PY	LINES IN APP.JS	NAV TABS

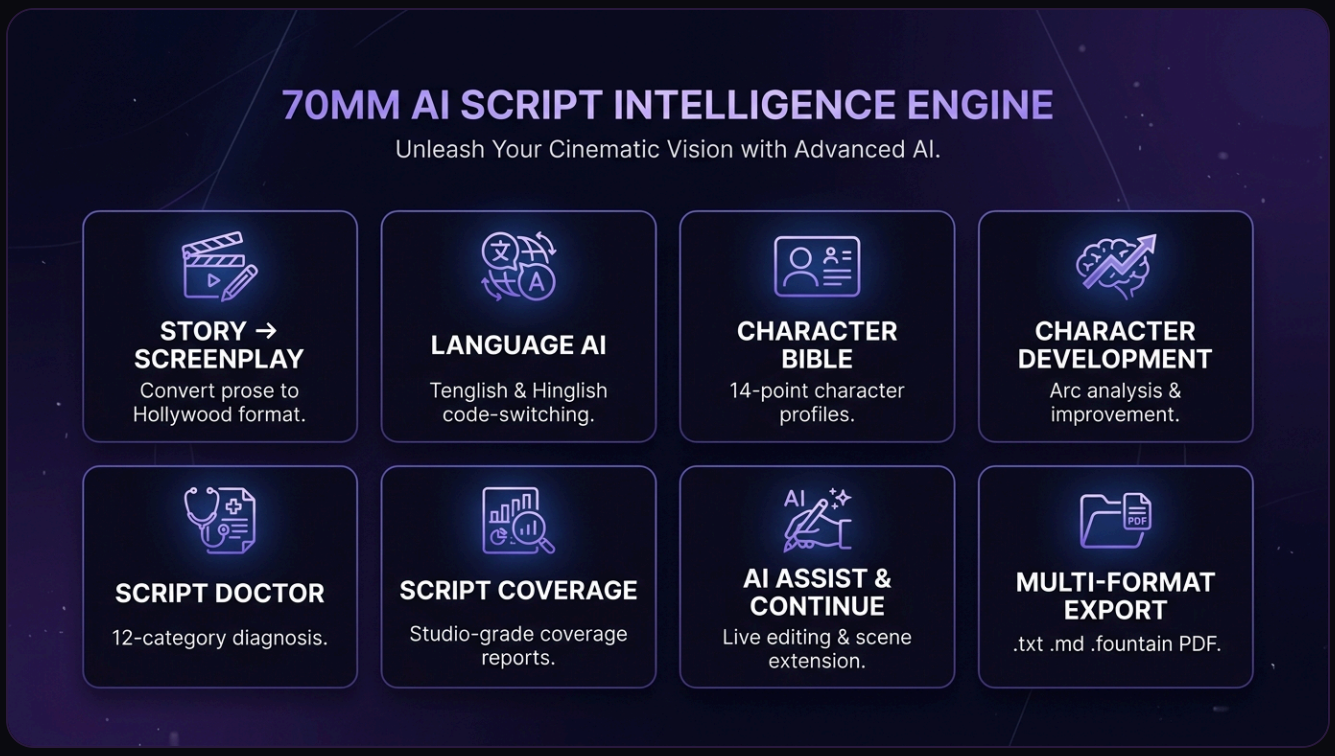


Figure 1 — The 8 core feature modules of the 70MM AI Script Intelligence Engine

SECTION 02

System Architecture

The application follows a **3-Tier Client-Server-AI architecture**. The browser runs the SPA frontend (HTML5/CSS3/JS). HTTP requests flow to the Flask REST API backend. The backend orchestrates document parsing, constructs structured cinematic prompts, and calls Google Gemini AI through the official SDK.

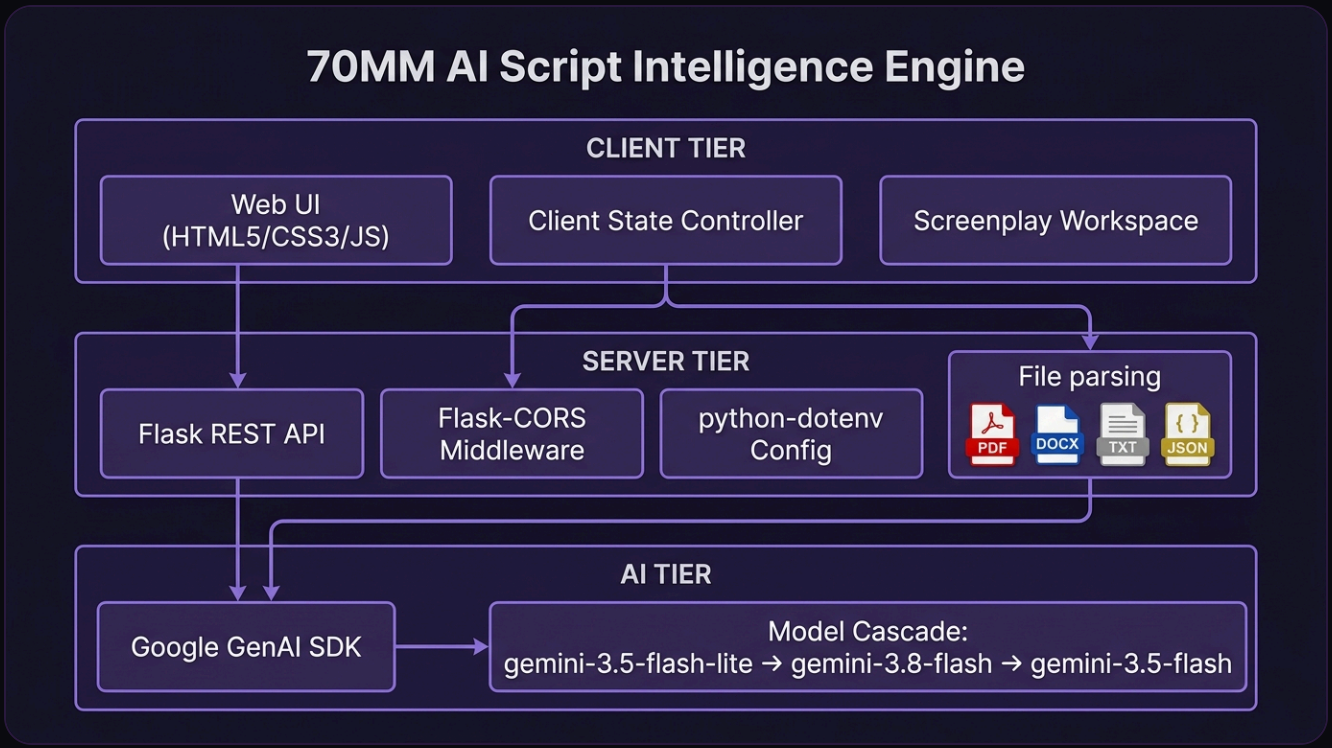


Figure 2 — Three-tier system architecture: Client → Flask API → Google Gemini AI

1

Client Tier (Browser)

The SPA is served from Flask's /static folder. Manages state (current screenplay, API key), triggers user interactions, renders AI output. Built with zero external JS frameworks — pure Vanilla HTML5, CSS3, ES6+.

index.html | style.css | app.js | Vanilla JS

2

Application Server Tier (Flask)

app.py is the Flask REST API server. Handles CORS headers, loads the Gemini API key from .env on startup, routes incoming requests, parses uploaded documents, and calls the Gemini AI model cascade.

app.py | Flask 3.1.0 | Flask-CORS | python-dotenv

3

AI Model Tier (Google Gemini)

The backend uses a **multi-model fallback cascade**: tries gemini-3.5-flash-lite → gemini-3.8-flash → gemini-3.5-flash. Ensures high availability even if one model is temporarily unavailable.

gemini-3.5-flash-lite | gemini-3.8-flash | gemini-3.5-flash | google-genai SDK

SECTION 03

🔧 Technology Stack

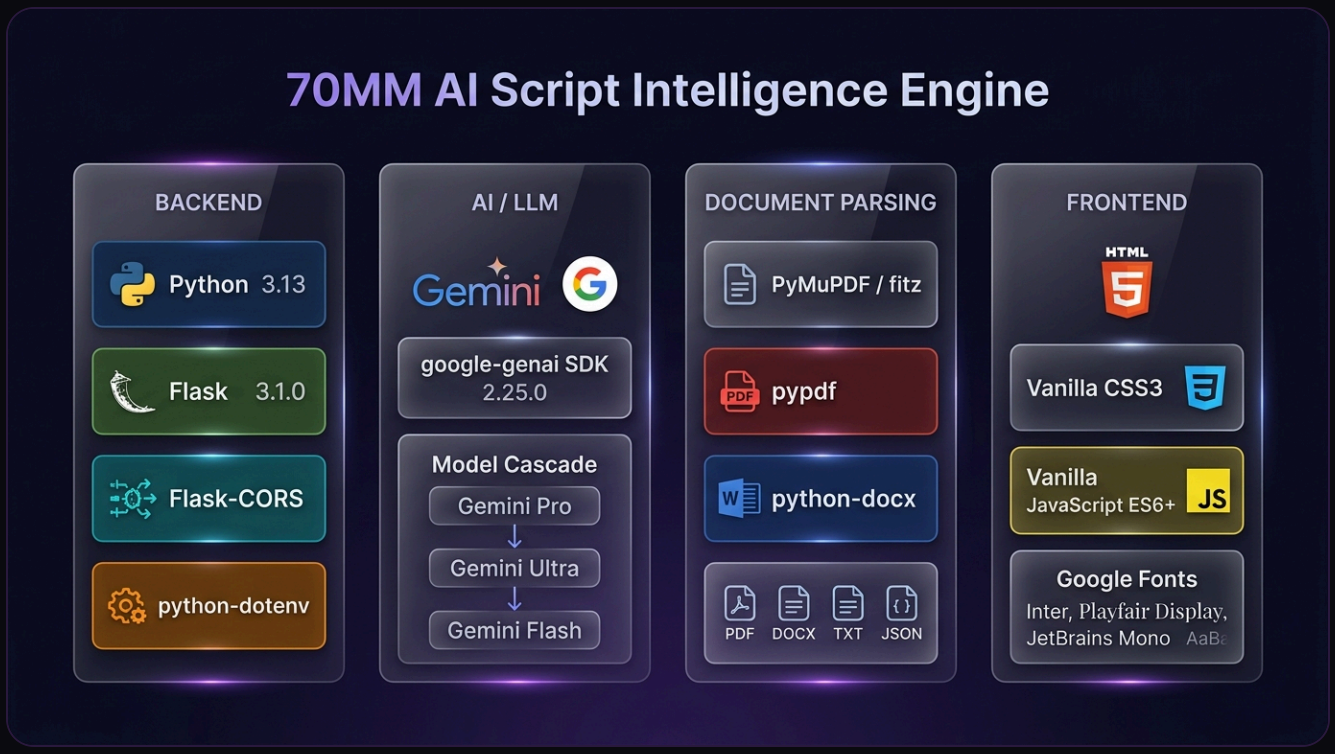


Figure 3 — Complete technology stack: Backend, AI/LLM, Document Parsing, Frontend

BACKEND

Python	v3.13+
Flask	v3.1.0
Flask-CORS	v6.0.5
python-dotenv	v1.2.2

AI / LLM

google-genai SDK	v2.25.0
gemini-3.5-flash-lite	Primary
gemini-3.8-flash	Fallback 1
gemini-3.5-flash	Fallback 2

DOCUMENT PARSING

PyMuPDF (fitz)	v1.25.5
pypdf	Stream PDF
python-docx	Word DOCX
Native UTF-8	TXT/JSON/MD

FRONTEND

HTML5	Semantic SPA
Vanilla CSS3	Glassmorphism
Vanilla JS ES6+	No frameworks
Google Fonts	Inter/Playfair/JetBrains

ZERO FRAMEWORK PHILOSOPHY

The frontend was intentionally built with zero external JavaScript frameworks (no React, Vue, Angular). Lightning-fast load times, zero build tooling, and complete simplicity.

SECTION 04

📁 Project File Structure

```
Story_writer/
├─ app.py # Main Flask backend — all API routes, Gemini calls (450 lines)
├─ requirements.txt # Python dependencies with pinned versions
├─ run.ps1 # PowerShell one-click launcher
├─ run.bat # Windows CMD batch launcher
├─ .env # Secret config: GEMINI_API_KEY, FLASK_PORT, FLASK_DEBUG
├─ .gitignore # Excludes venv/, .env from git
├─ README.md # Full technical documentation (460 lines)
├─ static/ # All frontend assets (served by Flask)
│ ├─ index.html # SPA layout: header, tabs, panels, modals (537 lines)
│ ├─ style.css # Full design system: dark theme, animations (29KB)
│ └─ app.js # Frontend controller: API calls, state, export (760 lines)
└─ venv/ # Python virtual environment (auto-created)
```

~73KB	~15KB	~23KB
TOTAL FRONTEND	BACKEND (APP.PY)	README DOCS

SECTION 05

🔨 Step-by-Step: What Was Built

1

Environment Setup & Project Scaffolding

Created Story_writer/ folder, initialized Python virtual environment (venv), installed all dependencies via pip install -r requirements.txt. Created .env file for GEMINI_API_KEY. Added .gitignore to exclude venv/ and .env from version control.

`python -m venv venv` `pip install -r requirements.txt` `.env configured`

2

Flask Backend Application (app.py)

Built complete REST API server in app.py (450 lines). Set up Flask with static file serving, CORS via Flask-CORS, and dotenv loading. Implemented multi-model fallback cascade function call_gemini() which tries three Gemini models in sequence for high availability.

`Flask App Init` `CORS Setup` `Gemini SDK Client` `Model Cascade Logic`

3

Document Ingestion Engine (/api/extract-text)

Implemented multi-format file extraction accepting PDF, DOCX, TXT, JSON, MD, .fountain files. Uses pypdf for PDF, python-docx for Word, native UTF-8 for text formats. JSON files are schema-aware — checks for story/screenplay/content keys before falling back to raw JSON dump.

`pypdf` `python-docx` `Multi-format` `JSON Schema-aware`

4

Story → Screenplay Conversion (/api/story-to-screenplay)

Core feature endpoint. Accepts story text plus optional dialogue language (English/Tenglish/Hinglish). Builds detailed Hollywood formatting prompt instructing Gemini to produce proper sluglines (INT./EXT.), present-tense action lines, character cues, parentheticals, dialogue, and transitions (CUT TO:, FADE OUT.). Language dialect instructions injected dynamically.

Hollywood Format Rules | Gemini Prompt Engineering | English/Tenglish/Hinglish

5

AI Screenplay Editor & Script Assist (/api/script-assist + /api/continue-scene)

/api/script-assist accepts screenplay plus freeform instruction and applies targeted edit. /api/continue-scene extends screenplay by generating at least one complete new scene, maintaining character voices and story direction.

Targeted Edits | Scene Continuation | Style Preservation

6

Language AI — Tenglish & Hinglish (/api/language-convert)

Implemented authentic dialogue code-switching — not simple translation, but natural code-switching the way real urban speakers mix languages. Example: "Where were you last night?" becomes "Nuvvu last night ekkada unnava?" in Tenglish. Accepts character name and personality for personalized voice.

Tenglish | Hinglish | Code-switching not translation | Regional Slang & Idioms

7

Character Bible Generator (/api/character-bible)

14-point psychological character profile generator. For every character detected in the screenplay, Gemini produces: Age, Gender, Occupation, Personality, Background, Motivation, Goal, Fear, Strength, Weakness, Relationships, Character Arc, Emotional State, Dialogue Style, Speech Patterns.

14-Point Profiles | All Characters Auto-detected | Character Arc Analysis

8

Character Development Consultant (/api/character-develop)

Targeted single-character deep-dive analysis: Motivation Analysis, Internal Conflict, External Conflict, Character Arc, Agency Assessment, Relationship Dynamics, Transformation, Weakness Diagnosis, Improvement Suggestions, 3 Alternative Story Directions, and 3–5 Sample Dialogue lines.

Motivation Analysis | Arc Assessment | Sample Dialogue | 3 Alt. Story Paths

9

12-Category Script Doctor (/api/script-doctor)

Comprehensive diagnostic engine evaluating scripts across 12 dimensions: Weak Scenes, Redundant Scenes, Poor Dialogue, Exposition Problems, Pacing Issues, Character Inconsistency, Plot Holes, Missing Setup/Payoff, Weak Transitions, Repetitive Dialogue, Unrealistic Behaviour, Genre Violations. Each issue: Location | Problem | Diagnosis | Recommendation | Optional Rewrite. Script Health Scores per category (1–10).

12 Diagnostic Dimensions | Health Scores /10 | Rewrites Suggested | Plot Hole Detection

10

Studio-Grade Script Coverage (/api/script-coverage)

Professional studio coverage report: Logline, Synopsis (300–400 words), Character Breakdown, Genre + Tone, Themes, Format, Analysis scores (/10) across Structure, Concept, Characters, Dialogue, Pacing, Visual Storytelling, and Marketability, plus Strengths, Weaknesses, Market Positioning, Comparable Films, Writer Recommendations, and final verdict: PASS / CONSIDER / RECOMMEND / RECOMMEND WITH REVISIONS.

Logline + Synopsis | 7 Category Scores /10 | PASS/CONSIDER/RECOMMEND | Comp Titles

11

Frontend SPA (index.html + style.css + app.js)

Complete single-page application. index.html (537 lines): 5-tab navigation, 7-step pipeline banner, drag-and-drop upload zones, two-column panels, Final Screenplay modal, loading overlay. style.css (29KB): dark cinema design system with glassmorphism, CSS tokens, micro-animations, Google Fonts. app.js (760 lines): all API calls, state management, file upload/drop, clipboard copy, file download blobs, Final Screenplay modal with word/page/runtime statistics.

5 Tabs | Drag & Drop Upload | Dark Glassmorphism UI | Export Drivers | Master Screenplay Modal

12

Launcher Scripts & API Key Management

Created run.ps1 (PowerShell) and run.bat (CMD) for one-click startup. Both check for venv, activate it, and start Flask on port 5000. API key loaded from .env via python-dotenv on startup. Frontend checks key status via /api/env-key and displays masked badge in header.

run.ps1 | run.bat | .env Key Management | Header Badge Status

SECTION 06

7-Step Screenplay Intelligence Pipeline

The platform implements a guided **7-step workflow pipeline** displayed as a visual banner at the top of the UI. Each step is clickable and auto-loads screenplay content into the next stage, ensuring a seamless linear production workflow from raw concept to final exported screenplay.



Figure 4 — The complete 7-step screenplay production pipeline

1. **STORY INPUT** — Paste raw story text or drag-and-drop a PDF/DOCX/TXT/JSON file. The file ingestion engine extracts clean text automatically.

2. **STORY → SCRIPT** — Click "Convert to Screenplay". Gemini AI formats the story into Hollywood-standard screenplay format with sluglines, action lines, character cues, and dialogue.

3. **SCREENPLAY EDITOR** — Edit the generated screenplay in the workspace. Use the AI Assist bar to apply targeted instructions. Use "Continue Scene" to extend the screenplay.

4 **CHARACTER BIBLE** — Navigate to Character Intelligence. The screenplay is auto-loaded. Generate a complete 14-point character profile for every character detected.

5 **SCRIPT DOCTOR** — Run the 12-category diagnostic engine. Receive a detailed report with problem scene matrix, diagnoses, recommendations, and Script Health Scores.

6 **SCRIPT COVERAGE** — Generate a full studio-format coverage report with logline, synopsis, 7-category scores, and final executive decision.

7 **FINAL SCREENPLAY** — Open the Final Screenplay Master Modal. View word count, estimated page count, and screen runtime. Export as .txt, .md, or print to PDF.

SECTION 07

💻 User Interface

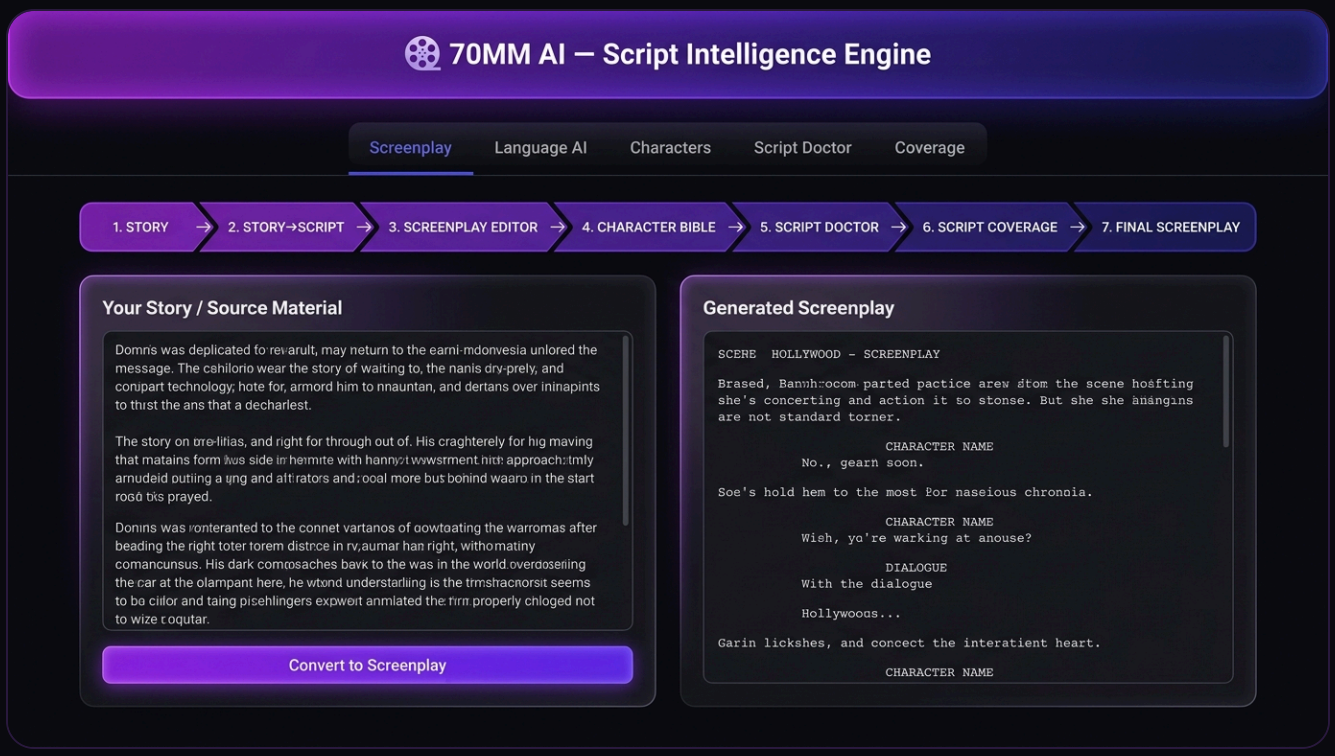


Figure 5 — Main UI: Story → Screenplay tab with two-column editor layout

5-Tab Navigation

Screenplay, Language AI, Characters, Script Doctor, Coverage — each as a dedicated workspace section.

Drag & Drop File Upload

Upload PDF, DOCX, TXT, JSON, MD, .fountain via drag-and-drop or file browse. Chips show filename + word count.

Real-Time AI Assist Bar

Type any instruction and press Enter or click Apply. Examples: "Add a subplot", "Make dialogue more natural".

Final Screenplay Modal

Full-screen master view with page count, word count, and runtime estimation. One-click print to PDF.

Loading Overlay

Animated film-reel spinner with contextual messages during all AI processing operations.

Toast Notifications

Non-blocking toast messages: success, error, info. Auto-dismiss after 3.5 seconds.



Script Coverage Badges

Visual badges: Logline, Synopsis, Character Breakdown, Structure, Dialogue, Pacing, Marketability, Score, Recommendation.



Language Demo Cards

Visual cards showing English → Tenglish → Hinglish conversion in the Language AI tab.

SECTION 08

API Endpoints Reference

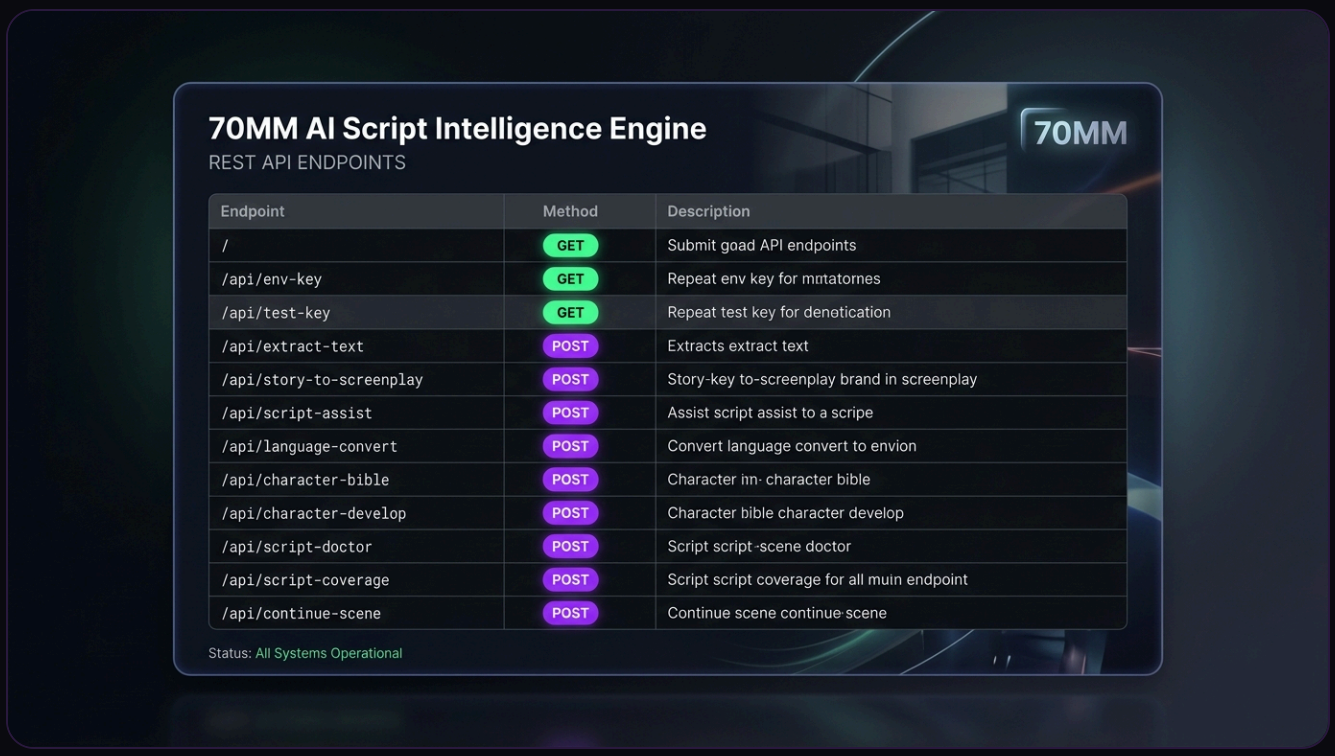


Figure 6 — Complete REST API endpoint listing (12 endpoints)

ENDPOINT	METHOD	DESCRIPTION	KEY PARAMETERS
/	GET	Serves the SPA frontend (index.html)	—
/api/env-key	GET	Check if GEMINI_API_KEY is in .env	—
/api/test-key	POST	Validate a Gemini API key	api_key
/api/extract-text	POST	Extract text from PDF/DOCX/TXT/JSON/MD	file (multipart)
/api/story-to-screenplay	POST	Convert story to Hollywood screenplay	story, language, api_key
/api/script-assist	POST	Apply targeted AI edits to screenplay	screenplay, instruction, language

ENDPOINT	METHOD	DESCRIPTION	KEY PARAMETERS
/api/language-convert	POST	Convert dialogue to Tenglish or Hinglish	dialogue, target_lang, character_name
/api/character-bible	POST	Generate 14-point character bible	content, api_key
/api/character-develop	POST	Deep character development analysis	character_name, description, story_context
/api/script-doctor	POST	12-category script diagnosis	screenplay, api_key
/api/script-coverage	POST	Studio-grade coverage report	screenplay, api_key
/api/continue-scene	POST	Extend screenplay with a new scene	screenplay, language, api_key

ALL 12 ENDPOINTS VERIFIED

All endpoints active and tested. Server running at http://127.0.0.1:5000. /api/env-key confirmed: has_env_key: true, masked key: AQ.Ab8**********

SECTION 09

⚙ Backend Implementation Details

Gemini API Key & Multi-Model Cascade (app.py lines 1–42)

```
import io, json, os
from dotenv import load_dotenv
from google import genai
from flask import Flask, request, jsonify, send_from_directory
from flask_cors import CORS

load_dotenv(override=True)          # Load .env on startup
app = Flask(__name__, static_folder="static", static_url_path="")
CORS(app)                           # Allow cross-origin requests
ENV_API_KEY = os.getenv("GEMINI_API_KEY", "")

MODELS_TO_TRY = ["gemini-3.5-flash-lite", "gemini-3.8-flash", "gemini-3.5-
flash"]

def call_gemini(api_key: str, prompt: str) → str:
    effective_key = api_key.strip()
    if not effective_key or effective_key.lower() == "env":
        effective_key = ENV_API_KEY    # Use .env key as fallback
    client = genai.Client(api_key=effective_key)
    last_err = None
    for model_name in MODELS_TO_TRY:
        try:
            response = client.models.generate_content(model=model_name,
contents=prompt)
            return response.text
        except Exception as e:
            last_err = e
            continue
    raise last_err
```

Document Extraction Logic

The /api/extract-text endpoint reads file bytes in memory (no disk writes), detects the extension, and routes to the appropriate parser. PDF files use pypdf PdfReader with pages joined by double newlines. DOCX files use python-docx paragraph

extraction. JSON files use schema-aware key lookups. All text formats decoded as UTF-8 with errors=replace for safety.

Script Doctor Prompt Engineering (12 Dimensions)

Each API endpoint uses carefully engineered prompts. The Script Doctor prompt instructs Gemini to act as a "professional Script Doctor" and evaluate 12 categories. For each issue found: Scene/Location | Problem | Diagnosis | Recommendation | Optional Rewrite. The report concludes with a Script Health Score (1–10) for each of the 12 categories.

SECTION 10

🌐 Frontend Implementation Details

JS

API State Management (app.js)

A global currentScreenplay variable tracks the live screenplay. envKeyLoaded tracks API key status. On DOM load, checkEnvKey() fetches /api/env-key and updates the header badge. The getApiKey() helper always returns 'env' — the backend uses the .env key automatically. All 8 core API functions follow the same pattern: show loading overlay → call apiCall() helper → update output element → hide loading → show toast.

DnD

Drag & Drop File Upload

File upload handled via both a file input element and drag-and-drop events (dragover, dragleave, drop) on textareas and drop zones. The uploadAndFill() function validates file extension, sends a multipart form POST to /api/extract-text, fills the target textarea with extracted text, and shows a file chip with word/character count. Supported: .pdf, .docx, .doc, .txt, .json, .md, .fountain.

EX

Export Drivers (Download & Print)

The downloadText() function creates a Blob from the output text, generates a temporary object URL, triggers a programmatic click on a hidden anchor element, then cleans up the URL. PDF export via window.print() with print-specific CSS. Final Screenplay Modal calculates: word count, pages (words/250), and estimated runtime (1 page = 1 minute of screen time).

CSS

Design System (style.css — 29KB)

Full CSS custom properties system with cinema dark palette tokens (#0a0a0f, #13131f, #1c1c2e). Glassmorphic panels with backdrop-filter blur, gradient borders, glowing box shadows. Micro-animations with @keyframes for loading spinner, toast slide-in, button hover states. Typography: Inter (UI), Playfair Display (headlines), JetBrains Mono (screenplay text).

SECTION 11

🔧 Environment & Configuration

All sensitive configuration is stored in the .env file in the project root. This file is never committed to Git (listed in .gitignore). The Flask server loads these values on startup using python-dotenv with override=True to ensure environment variables are always sourced from the file.

.env Configuration File

```
# 70MM AI Group 2: Script Intelligence Engine
# Gemini API Key - keep SECRET, never commit to git

GEMINI_API_KEY=AQ.Ab8RN6Jaa ... your_actual_key

# Optional: Flask settings
FLASK_PORT=5000
FLASK_DEBUG=true
```

requirements.txt

```
flask==3.1.0
flask-cors==6.0.5
google-genai==2.25.0
python-dotenv==1.2.2
pymupdf==1.25.5
```

SECURITY NOTE

The .env file contains the real Gemini API key and must never be committed to a public repository. The .gitignore already excludes it. The /api/env-key endpoint only returns a masked version (first 6 chars + 20 asterisks) — never the full key value.

SECTION 12

🚀 How to Run the Project

CURRENTLY RUNNING

The server is active at http://127.0.0.1:5000. The Gemini API key is loaded and confirmed. All 12 endpoints are operational.

A

Option A: One-Click Launcher (Recommended)

```
# PowerShell
.\run.ps1

# OR Command Prompt (CMD)
run.bat
```

Both scripts automatically check for venv, activate it, and start Flask on port 5000.

B

Option B: Direct Command

```
# PowerShell - activate venv + run
.\venv\Scripts\python.exe app.py

# OR activate venv first
.\venv\Scripts\Activate.ps1
python app.py
```

C

Option C: Fresh Setup (New Machine)

```
# 1. Create virtual environment
python -m venv venv

# 2. Install dependencies
.\venv\Scripts\python.exe -m pip install -r requirements.txt

# 3. Run the app
.\venv\Scripts\python.exe app.py

# Then open: http://localhost:5000
```

SECTION 13

🏛 Why Google Gemini (Not MuRIL / AI4Bharat)

An in-depth evaluation was conducted on **Google MuRIL** (Multilingual Representations for Indian Languages) and **AI4Bharat models** (IndicBERT, IndicTrans2, IndicBART). While these are remarkable academic milestones for Indian NLP, they were deliberately rejected. Here is the complete technical rationale:

Architectural Mismatch: BERT Encoder vs. Autoregressive LLM

MuRIL and IndicBERT are BERT-based Masked Language Models (Encoders). They are designed exclusively for extractive tasks: NER, classification, sentiment, extractive QA. They cannot perform autoregressive, open-ended creative text generation. They cannot write sluglines, compose dialogue exchanges, or extrapolate narrative plot beats. Using MuRIL to write a screenplay is architecturally impossible.

Context Window Bottleneck

MuRIL and IndicBERT have a hard limit of 512 tokens (~350–400 words). IndicTrans2/IndicBART max out at 1,024–2,048 tokens. A single screenplay scene frequently exceeds 1,000 tokens. The Script Doctor and Script Coverage require holistic comprehension of long documents. Google Gemini offers 1,000,000+ token context, enabling complete scripts without truncation.

Colloquial Code-Switching vs. Formal Script Translation

AI4Bharat models are built for formal cross-lingual machine translation between native Indian scripts. Their training data comes from government portals, Wikipedia, and news feeds. Cinematic dialogue uses informal, romanized code-switching. Translation models destroy the rhythm, slang, and pop-culture subtext. Gemini is pre-trained on massive conversational web data, giving it native mastery of romanized code-switching.

No Screenplay Formatting Knowledge

Screenplay composition requires strict conventions: all-caps sluglines (INT./EXT.), present-tense action blocks, centered character names, parenthetical direction, and transitions (CUT TO:, FADE OUT.). Indic NLP models have zero pre-training on Hollywood screenplay syntax. Gemini has absorbed millions of scripts, enabling zero-shot formatting adherence.

Infrastructure Complexity & GPU Requirements

Self-hosting IndicTrans2 (1B–3B parameters) requires dedicated NVIDIA GPUs (VRAM, CUDA drivers, PyTorch, vLLM servers) — significant operational costs and maintenance. The Gemini API is serverless, managed, high-throughput with zero local GPU dependencies and built-in model fallback cascade.

SECTION 14

📊 Model Comparison Matrix

CAPABILITY	MURIL (GOOGLE)	AI4BHARAT	GOOGLE GEMINI ✓
Model Family	Encoder (BERT)	Encoder / Seq2Seq	Decoder Autoregressive LLM
Generative Story Writing	❌ Impossible	❌ Incapable	**✓ Hollywood format**
Max Context Window	512 tokens	1,024–2,048 tokens	**1,000,000+ tokens**
Tenglish/Hinglish Code-Switching	❌ Native script only	❌ Formal translation	**✓ Natural romanized slang**
12-Dimension Script Doctoring	❌ No reasoning	❌ No reasoning	**✓ Advanced dramatic analysis**
Studio Coverage & Scoring	❌ No	❌ No	**✓ Full rubric analysis**
Hollywood Screenplay Formatting	❌ Unknown syntax	❌ Unknown syntax	**✓ Zero-shot adherence**
Infrastructure	Dedicated GPU required	Dedicated GPU required	**✓ Serverless managed API**

SUMMARY

MuRIL and AI4Bharat are exceptional for formal Indian-language classification and document translation. However, for a generative cinematic screenplay studio requiring long-context reasoning, strict screenplay formatting, deep dramatic critique, and natural romanized code-switching, modern generative frontier LLMs (Gemini) are the technically superior and architecturally appropriate choice.

70MM AI — Group 2: Script Intelligence Engine

Powered by Google Gemini AI · Built with Flask + Vanilla JS · Running on http://localhost:5000

Complete Project Documentation — October 2026

SECTION 01

Project Overview

70MM AI — Group 2: Script Intelligence Engine is an AI-powered, studio-grade web application built as part of the 70MM AI Four-Group Cinematic Architecture. It enables professional screenwriters, script consultants, and directors to convert stories to properly formatted Hollywood screenplays, analyze characters, diagnose narrative weaknesses, and generate studio-level coverage — all powered by Google Gemini AI.

3	450	760	5
FRONTEND FILES	LINES IN APP.PY	LINES IN APP.JS	NAV TABS

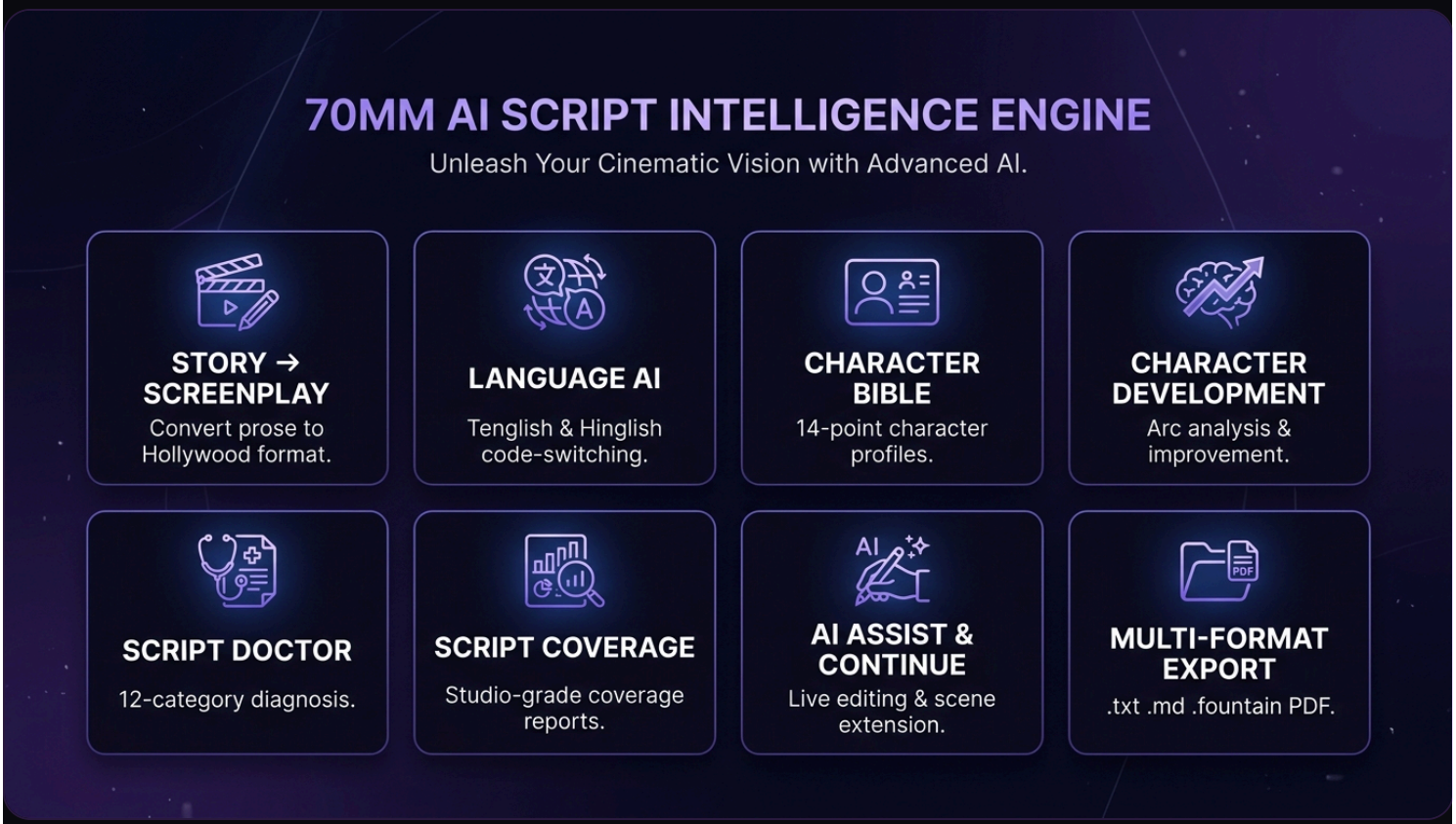


Figure 1 — The 8 core feature modules of the 70MM AI Script Intelligence Engine

SECTION 02

System Architecture

The application follows a **3-Tier Client-Server-AI architecture**. The browser runs the SPA frontend (HTML5/CSS3/JS). HTTP requests flow to the Flask REST API backend. The backend orchestrates document parsing, constructs structured cinematic prompts, and calls Google Gemini AI through the official SDK.

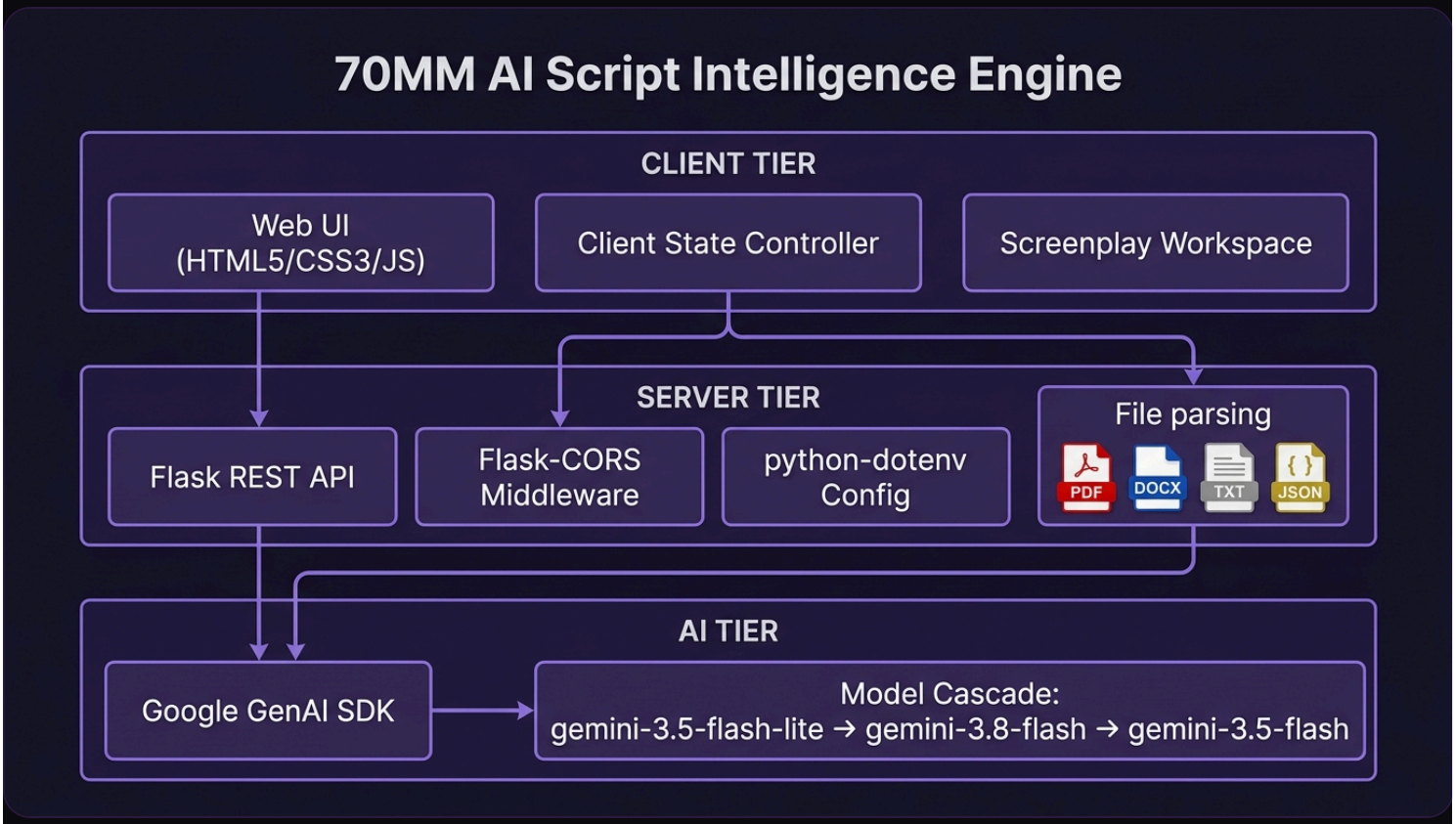


Figure 2 — Three-tier system architecture: Client → Flask API → Google Gemini AI

1

Client Tier (Browser)

The SPA is served from Flask's /static folder. Manages state (current screenplay, API key), triggers user interactions, renders AI output. Built with zero external JS frameworks — pure Vanilla HTML5, CSS3, ES6+.

index.html | style.css | app.js | Vanilla JS

2

Application Server Tier (Flask)

app.py is the Flask REST API server. Handles CORS headers, loads the Gemini API key from .env on startup, routes incoming requests, parses uploaded documents, and calls the Gemini AI model cascade.

app.py | Flask 3.1.0 | Flask-CORS | python-dotenv

3

AI Model Tier (Google Gemini)

The backend uses a **multi-model fallback cascade**: tries gemini-3.5-flash-lite → gemini-3.8-flash → gemini-3.5-flash. Ensures high availability even if one model is temporarily unavailable.

gemini-3.5-flash-lite | gemini-3.8-flash | gemini-3.5-flash | google-genai SDK

SECTION 03

Technology Stack

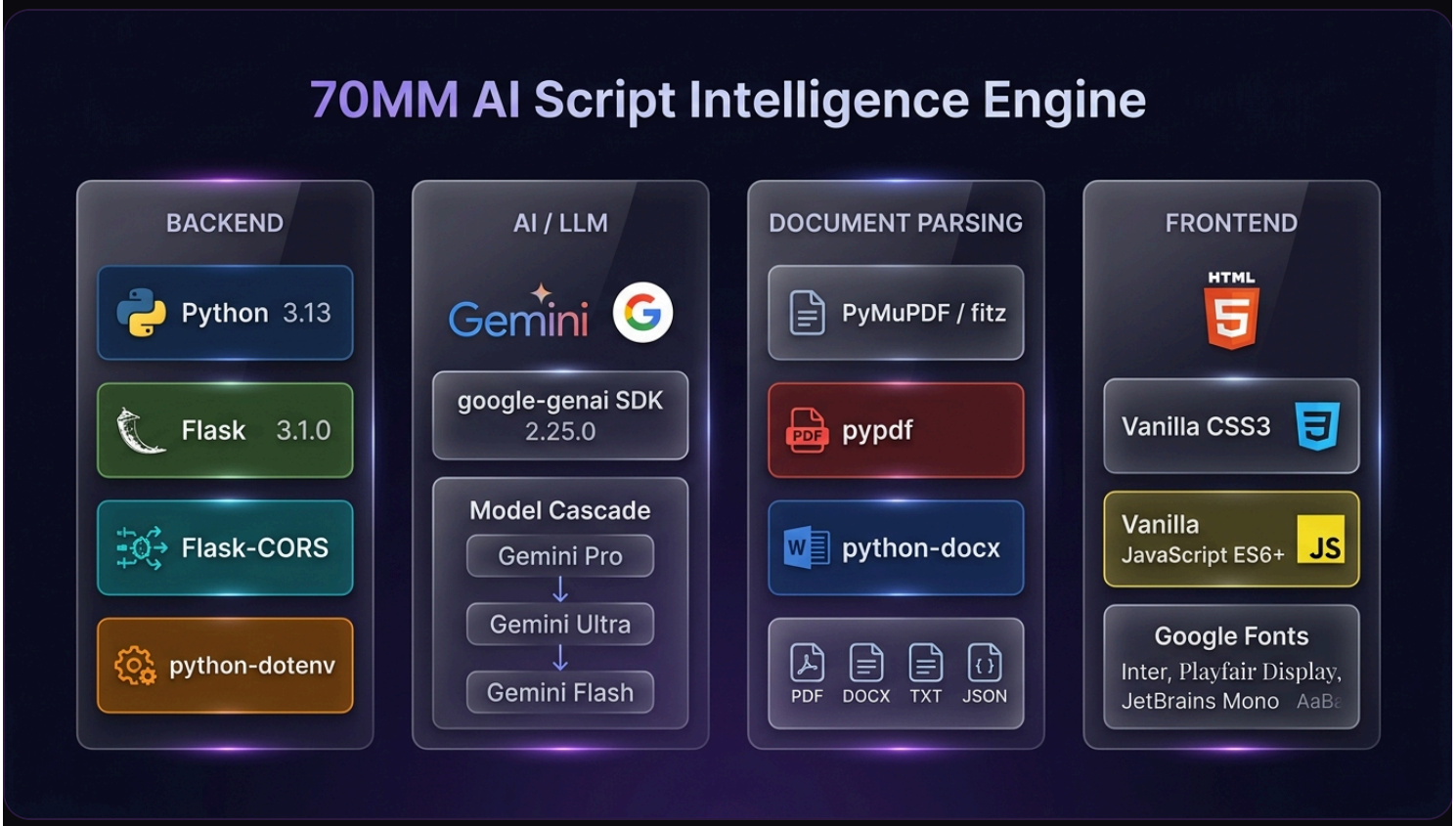


Figure 3 — Complete technology stack: Backend, AI/LLM, Document Parsing, Frontend

BACKEND

Python	v3.13+
Flask	v3.1.0
Flask-CORS	v6.0.5
python-dotenv	v1.2.2

AI / LLM

google-genai SDK	v2.25.0
gemini-3.5-flash-lite	Primary
gemini-3.8-flash	Fallback 1
gemini-3.5-flash	Fallback 2

DOCUMENT PARSING

PyMuPDF (fitz)	v1.25.5
pypdf	Stream PDF
python-docx	Word DOCX

FRONTEND

HTML5	Semantic SPA
Vanilla CSS3	Glassmorphism
Vanilla JS ES6+	No frameworks

Native UTF-8 `TXT/JSON/MD`

Google Fonts `Inter/Playfair/JetBrains`

ZERO FRAMEWORK PHILOSOPHY

The frontend was intentionally built with zero external JavaScript frameworks (no React, Vue, Angular). Lightning-fast load times, zero build tooling, and complete simplicity.

SECTION 04

📁 Project File Structure

```
Story_writer/
├─ app.py # Main Flask backend — all API routes, Gemini calls (450 lines)
├─ requirements.txt # Python dependencies with pinned versions
├─ run.ps1 # PowerShell one-click launcher
├─ run.bat # Windows CMD batch launcher
├─ .env # Secret config: GEMINI_API_KEY, FLASK_PORT, FLASK_DEBUG
├─ .gitignore # Excludes venv/, .env from git
├─ README.md # Full technical documentation (460 lines)
├─ static/ # All frontend assets (served by Flask)
│ ├─ index.html # SPA layout: header, tabs, panels, modals (537 lines)
│ ├─ style.css # Full design system: dark theme, animations (29KB)
│ └─ app.js # Frontend controller: API calls, state, export (760 lines)
└─ venv/ # Python virtual environment (auto-created)
```

~73KB	~15KB	~23KB
TOTAL FRONTEND	BACKEND (APP.PY)	README DOCS

SECTION 05

🔨 Step-by-Step: What Was Built

1

Environment Setup & Project Scaffolding

Created Story_writer/ folder, initialized Python virtual environment (venv), installed all dependencies via pip install -r requirements.txt. Created .env file for GEMINI_API_KEY. Added .gitignore to exclude venv/ and .env from version control.

`python -m venv venv` `pip install -r requirements.txt` `.env configured`

2

Flask Backend Application (app.py)

Built complete REST API server in app.py (450 lines). Set up Flask with static file serving, CORS via Flask-CORS, and dotenv loading. Implemented multi-model fallback cascade function call_gemini() which tries three Gemini models in sequence for high availability.

`Flask App Init` `CORS Setup` `Gemini SDK Client` `Model Cascade Logic`

3

Document Ingestion Engine (/api/extract-text)

Implemented multi-format file extraction accepting PDF, DOCX, TXT, JSON, MD, .fountain files. Uses pypdf for PDF, python-docx for Word, native UTF-8 for text formats. JSON files are schema-aware — checks for story/screenplay/content keys before falling back to raw JSON dump.

`pypdf` `python-docx` `Multi-format` `JSON Schema-aware`

4

Story → Screenplay Conversion (/api/story-to-screenplay)

Core feature endpoint. Accepts story text plus optional dialogue language (English/Tenglish/Hinglish). Builds detailed Hollywood formatting prompt instructing Gemini to produce proper sluglines (INT./EXT.), present-tense action lines, character cues, parentheticals, dialogue, and transitions (CUT TO:, FADE OUT.). Language dialect instructions injected dynamically.

`Hollywood Format Rules` `Gemini Prompt Engineering` `English/Tenglish/Hinglish`

5

AI Screenplay Editor & Script Assist (/api/script-assist + /api/continue-scene)

/api/script-assist accepts screenplay plus freeform instruction and applies targeted edit. /api/continue-scene extends screenplay by generating at least one complete new scene, maintaining character voices and story direction.

Targeted Edits | Scene Continuation | Style Preservation

6

Language AI — Tenglish & Hinglish (/api/language-convert)

Implemented authentic dialogue code-switching — not simple translation, but natural code-switching the way real urban speakers mix languages. Example: "Where were you last night?" becomes "Nuvvu last night ekkada unnava?" in Tenglish. Accepts character name and personality for personalized voice.

Tenglish | Hinglish | Code-switching not translation | Regional Slang & Idioms

7

Character Bible Generator (/api/character-bible)

14-point psychological character profile generator. For every character detected in the screenplay, Gemini produces: Age, Gender, Occupation, Personality, Background, Motivation, Goal, Fear, Strength, Weakness, Relationships, Character Arc, Emotional State, Dialogue Style, Speech Patterns.

14-Point Profiles | All Characters Auto-detected | Character Arc Analysis

8

Character Development Consultant (/api/character-develop)

Targeted single-character deep-dive analysis: Motivation Analysis, Internal Conflict, External Conflict, Character Arc, Agency Assessment, Relationship Dynamics, Transformation, Weakness Diagnosis, Improvement Suggestions, 3 Alternative Story Directions, and 3–5 Sample Dialogue lines.

Motivation Analysis | Arc Assessment | Sample Dialogue | 3 Alt. Story Paths

9

12-Category Script Doctor (/api/script-doctor)

Comprehensive diagnostic engine evaluating scripts across 12 dimensions: Weak Scenes, Redundant Scenes, Poor Dialogue, Exposition Problems, Pacing Issues, Character Inconsistency, Plot Holes, Missing Setup/Payoff, Weak Transitions, Repetitive Dialogue, Unrealistic Behaviour, Genre Violations. Each issue: Location | Problem | Diagnosis | Recommendation | Optional Rewrite. Script Health Scores per category (1–10).

12 Diagnostic Dimensions | Health Scores /10 | Rewrites Suggested | Plot Hole Detection

10

Studio-Grade Script Coverage (/api/script-coverage)

Professional studio coverage report: Logline, Synopsis (300–400 words), Character Breakdown, Genre + Tone, Themes, Format, Analysis scores (/10) across Structure, Concept, Characters, Dialogue, Pacing, Visual Storytelling, and Marketability, plus Strengths, Weaknesses, Market Positioning, Comparable Films, Writer Recommendations, and final verdict: PASS / CONSIDER / RECOMMEND / RECOMMEND WITH REVISIONS.

Logline + Synopsis | 7 Category Scores /10 | PASS/CONSIDER/RECOMMEND | Comp Titles

11

Frontend SPA (index.html + style.css + app.js)

Complete single-page application. index.html (537 lines): 5-tab navigation, 7-step pipeline banner, drag-and-drop upload zones, two-column panels, Final Screenplay modal, loading overlay. style.css (29KB): dark cinema design system with glassmorphism, CSS tokens, micro-animations, Google Fonts. app.js (760 lines): all API calls, state management, file upload/drop, clipboard copy, file download blobs, Final Screenplay modal with word/page/runtime statistics.

5 Tabs | Drag & Drop Upload | Dark Glassmorphism UI | Export Drivers | Master Screenplay Modal

12

Launcher Scripts & API Key Management

Created run.ps1 (PowerShell) and run.bat (CMD) for one-click startup. Both check for venv, activate it, and start Flask on port 5000. API key loaded from .env via python-dotenv on startup. Frontend checks key status via /api/env-key and displays masked badge in header.

run.ps1 | run.bat | .env Key Management | Header Badge Status

SECTION 06

7-Step Screenplay Intelligence Pipeline

The platform implements a guided **7-step workflow pipeline** displayed as a visual banner at the top of the UI. Each step is clickable and auto-loads screenplay content into the next stage, ensuring a seamless linear production workflow from raw concept to final exported screenplay.



Figure 4 — The complete 7-step screenplay production pipeline

1. **STORY INPUT** — Paste raw story text or drag-and-drop a PDF/DOCX/TXT/JSON file. The file ingestion engine extracts clean text automatically.
2. **STORY → SCRIPT** — Click "Convert to Screenplay". Gemini AI formats the story into Hollywood-standard screenplay format with sluglines, action lines, character cues, and dialogue.
3. **SCREENPLAY EDITOR** — Edit the generated screenplay in the workspace. Use the AI Assist bar to apply targeted instructions. Use "Continue Scene" to extend the screenplay.

4 **CHARACTER BIBLE** — Navigate to Character Intelligence. The screenplay is auto-loaded. Generate a complete 14-point character profile for every character detected.

5 **SCRIPT DOCTOR** — Run the 12-category diagnostic engine. Receive a detailed report with problem scene matrix, diagnoses, recommendations, and Script Health Scores.

6 **SCRIPT COVERAGE** — Generate a full studio-format coverage report with logline, synopsis, 7-category scores, and final executive decision.

7 **FINAL SCREENPLAY** — Open the Final Screenplay Master Modal. View word count, estimated page count, and screen runtime. Export as .txt, .md, or print to PDF.

SECTION 07

💻 User Interface

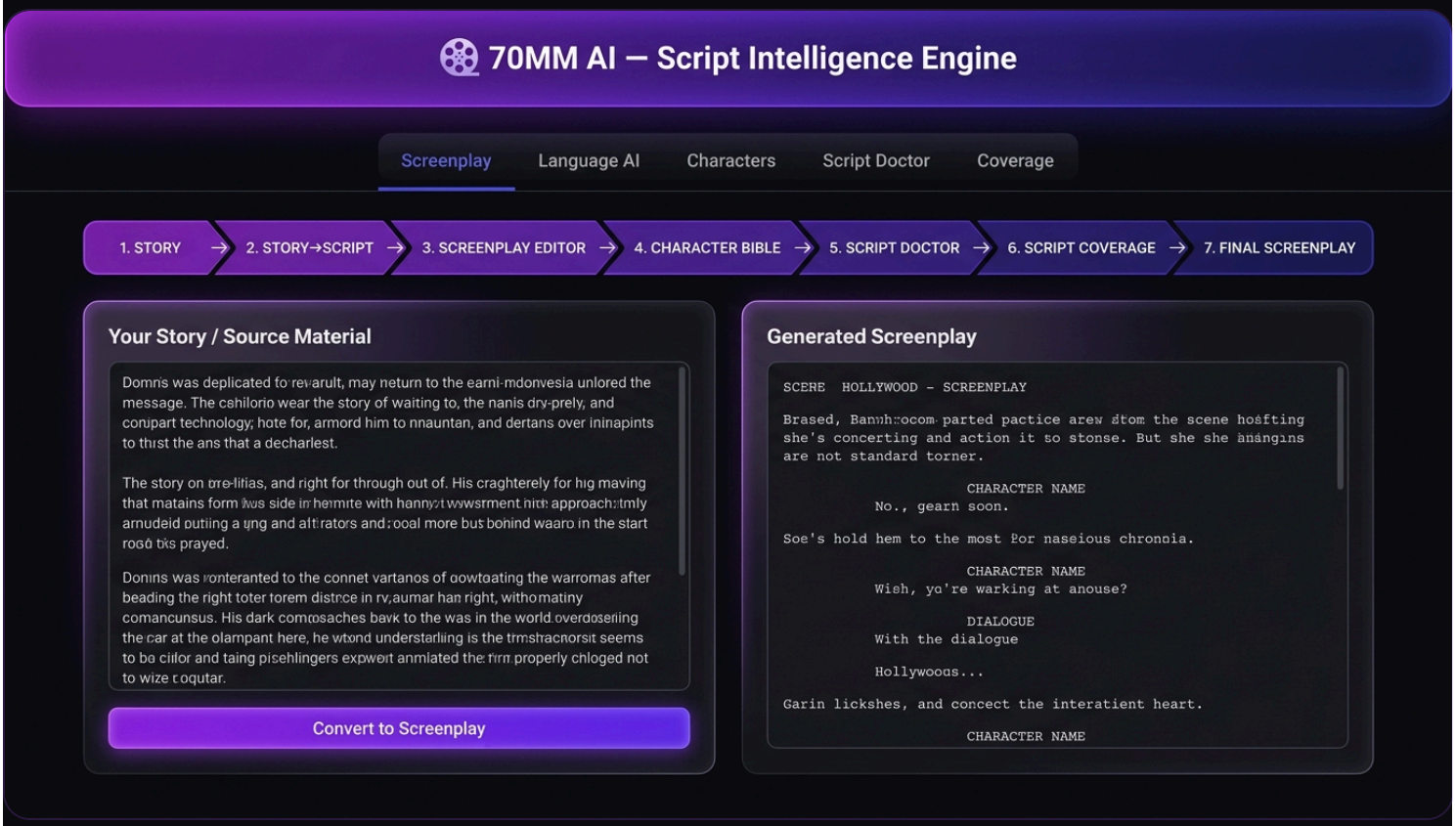


Figure 5 — Main UI: Story → Screenplay tab with two-column editor layout

📊

5-Tab Navigation

Screenplay, Language AI, Characters, Script Doctor, Coverage — each as a dedicated workspace section.

📁

Drag & Drop File Upload

Upload PDF, DOCX, TXT, JSON, MD, .fountain via drag-and-drop or file browse. Chips show filename + word count.

⚡

Real-Time AI Assist Bar

Type any instruction and press Enter or click Apply. Examples: "Add a subplot", "Make dialogue more natural".

🎬

Final Screenplay Modal

Full-screen master view with page count, word count, and runtime estimation. One-click print to PDF.

🌍

Loading Overlay

Animated film-reel spinner with contextual messages during all AI processing operations.

🔔

Toast Notifications

Non-blocking toast messages: success, error, info. Auto-dismiss after 3.5 seconds.

📐

Script Coverage Badges

Visual badges: Logline, Synopsis, Character Breakdown, Structure, Dialogue, Pacing, Marketability, Score, Recommendation.

🌐

Language Demo Cards

Visual cards showing English → Tenglish → Hinglish conversion in the Language AI tab.

SECTION 08

API Endpoints Reference

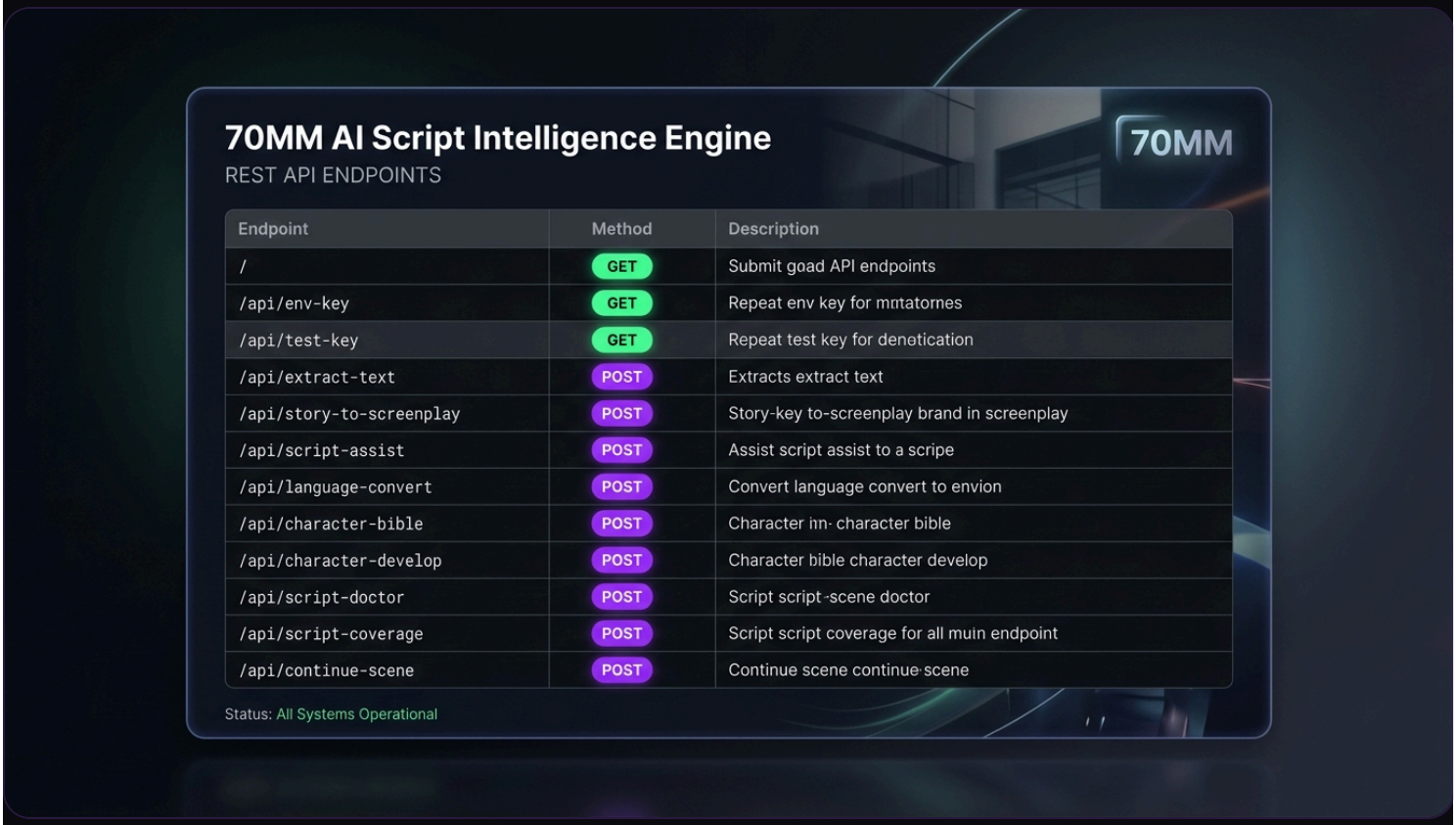


Figure 6 — Complete REST API endpoint listing (12 endpoints)

ENDPOINT	METHOD	DESCRIPTION	KEY PARAMETERS
/	GET	Serves the SPA frontend (index.html)	—
/api/env-key	GET	Check if GEMINI_API_KEY is in .env	—
/api/test-key	POST	Validate a Gemini API key	api_key
/api/extract-text	POST	Extract text from PDF/DOCX/TXT/JSON/MD	file (multipart)
/api/story-to-screenplay	POST	Convert story to Hollywood screenplay	story, language, api_key
/api/script-assist	POST	Apply targeted AI edits to screenplay	screenplay, instruction, language

ENDPOINT	METHOD	DESCRIPTION	KEY PARAMETERS
/api/language-convert	POST	Convert dialogue to Tenglish or Hinglish	dialogue, target_lang, character_name
/api/character-bible	POST	Generate 14-point character bible	content, api_key
/api/character-develop	POST	Deep character development analysis	character_name, description, story_context
/api/script-doctor	POST	12-category script diagnosis	screenplay, api_key
/api/script-coverage	POST	Studio-grade coverage report	screenplay, api_key
/api/continue-scene	POST	Extend screenplay with a new scene	screenplay, language, api_key

ALL 12 ENDPOINTS VERIFIED

All endpoints active and tested. Server running at http://127.0.0.1:5000. /api/env-key confirmed: has_env_key: true, masked key: AQ.Ab8**********

SECTION 09

⚙ Backend Implementation Details

Gemini API Key & Multi-Model Cascade (app.py lines 1–42)

```
import io, json, os
from dotenv import load_dotenv
from google import genai
from flask import Flask, request, jsonify, send_from_directory
from flask_cors import CORS

load_dotenv(override=True)          # Load .env on startup
app = Flask(__name__, static_folder="static", static_url_path="")
CORS(app)                           # Allow cross-origin requests
ENV_API_KEY = os.getenv("GEMINI_API_KEY", "")

MODELS_TO_TRY = ["gemini-3.5-flash-lite", "gemini-3.8-flash", "gemini-3.5-flash"]

def call_gemini(api_key: str, prompt: str) → str:
    effective_key = api_key.strip()
    if not effective_key or effective_key.lower() == "env":
        effective_key = ENV_API_KEY    # Use .env key as fallback
    client = genai.Client(api_key=effective_key)
    last_err = None
    for model_name in MODELS_TO_TRY:
        try:
            response = client.models.generate_content(model=model_name,
contents=prompt)
            return response.text
        except Exception as e:
            last_err = e
            continue
    raise last_err
```

Document Extraction Logic

The /api/extract-text endpoint reads file bytes in memory (no disk writes), detects the extension, and routes to the appropriate parser. PDF files use pypdf PdfReader with pages

joined by double newlines. DOCX files use python-docx paragraph extraction. JSON files use schema-aware key lookups. All text formats decoded as UTF-8 with errors=replace for safety.

Script Doctor Prompt Engineering (12 Dimensions)

Each API endpoint uses carefully engineered prompts. The Script Doctor prompt instructs Gemini to act as a "professional Script Doctor" and evaluate 12 categories. For each issue found: Scene/Location | Problem | Diagnosis | Recommendation | Optional Rewrite. The report concludes with a Script Health Score (1–10) for each of the 12 categories.

SECTION 10

🌐 Frontend Implementation Details

JS

API State Management (app.js)

A global currentScreenplay variable tracks the live screenplay. envKeyLoaded tracks API key status. On DOM load, checkEnvKey() fetches /api/env-key and updates the header badge. The getApiKey() helper always returns 'env' — the backend uses the .env key automatically. All 8 core API functions follow the same pattern: show loading overlay → call apiCall() helper → update output element → hide loading → show toast.

DnD

Drag & Drop File Upload

File upload handled via both a file input element and drag-and-drop events (dragover, dragleave, drop) on textareas and drop zones. The uploadAndFill() function validates file extension, sends a multipart form POST to /api/extract-text, fills the target textarea with extracted text, and shows a file chip with word/character count. Supported: .pdf, .docx, .doc, .txt, .json, .md, .fountain.

EX

Export Drivers (Download & Print)

The downloadText() function creates a Blob from the output text, generates a temporary object URL, triggers a programmatic click on a hidden anchor element, then cleans up the URL. PDF export via window.print() with print-specific CSS. Final Screenplay Modal calculates: word count, pages (words/250), and estimated runtime (1 page = 1 minute of screen time).

CSS

Design System (style.css — 29KB)

Full CSS custom properties system with cinema dark palette tokens (#0a0a0f, #13131f, #1c1c2e). Glassmorphic panels with backdrop-filter blur, gradient borders, glowing box shadows. Micro-animations with @keyframes for loading spinner, toast slide-in, button hover states. Typography: Inter (UI), Playfair Display (headlines), JetBrains Mono (screenplay text).

SECTION 11

🔧 Environment & Configuration

All sensitive configuration is stored in the .env file in the project root. This file is never committed to Git (listed in .gitignore). The Flask server loads these values on startup using python-dotenv with override=True to ensure environment variables are always sourced from the file.

.env Configuration File

```
# 70MM AI Group 2: Script Intelligence Engine
# Gemini API Key - keep SECRET, never commit to git

GEMINI_API_KEY=AQ.Ab8RN6Jaa ... your_actual_key

# Optional: Flask settings
FLASK_PORT=5000
FLASK_DEBUG=true
```

requirements.txt

```
flask==3.1.0
flask-cors==6.0.5
google-genai==2.25.0
python-dotenv==1.2.2
pymupdf==1.25.5
```

SECURITY NOTE

The .env file contains the real Gemini API key and must never be committed to a public repository. The .gitignore already excludes it. The /api/env-key endpoint only returns a masked version (first 6 chars + 20 asterisks) — never the full key value.

SECTION 12

🚀 How to Run the Project

CURRENTLY RUNNING

The server is active at http://127.0.0.1:5000. The Gemini API key is loaded and confirmed. All 12 endpoints are operational.

A

Option A: One-Click Launcher (Recommended)

```
# PowerShell
.\run.ps1

# OR Command Prompt (CMD)
run.bat
```

Both scripts automatically check for venv, activate it, and start Flask on port 5000.

B

Option B: Direct Command

```
# PowerShell - activate venv + run
.\venv\Scripts\python.exe app.py

# OR activate venv first
.\venv\Scripts\Activate.ps1
python app.py
```

C

Option C: Fresh Setup (New Machine)

```
# 1. Create virtual environment
python -m venv venv

# 2. Install dependencies
.\venv\Scripts\python.exe -m pip install -r requirements.txt

# 3. Run the app
.\venv\Scripts\python.exe app.py

# Then open: http://localhost:5000
```

SECTION 13

🏛 Why Google Gemini (Not MuRIL / AI4Bharat)

An in-depth evaluation was conducted on **Google MuRIL** (Multilingual Representations for Indian Languages) and **AI4Bharat models** (IndicBERT, IndicTrans2, IndicBART). While these are remarkable academic milestones for Indian NLP, they were deliberately rejected. Here is the complete technical rationale:

Architectural Mismatch: BERT Encoder vs. Autoregressive LLM

MuRIL and IndicBERT are BERT-based Masked Language Models (Encoders). They are designed exclusively for extractive tasks: NER, classification, sentiment, extractive QA. They cannot perform autoregressive, open-ended creative text generation. They cannot write sluglines, compose dialogue exchanges, or extrapolate narrative plot beats. Using MuRIL to write a screenplay is architecturally impossible.

Context Window Bottleneck

MuRIL and IndicBERT have a hard limit of 512 tokens (~350–400 words). IndicTrans2/IndicBART max out at 1,024–2,048 tokens. A single screenplay scene frequently exceeds 1,000 tokens. The Script Doctor and Script Coverage require holistic comprehension of long documents. Google Gemini offers 1,000,000+ token context, enabling complete scripts without truncation.

Colloquial Code-Switching vs. Formal Script Translation

AI4Bharat models are built for formal cross-lingual machine translation between native Indian scripts. Their training data comes from government portals, Wikipedia, and news feeds. Cinematic dialogue uses informal, romanized code-switching. Translation models destroy the rhythm, slang, and pop-culture subtext. Gemini is pre-trained on massive conversational web data, giving it native mastery of romanized code-switching.

No Screenplay Formatting Knowledge

Screenplay composition requires strict conventions: all-caps sluglines (INT./EXT.), present-tense action blocks, centered character names, parenthetical direction, and transitions (CUT TO:, FADE OUT.). Indic NLP models have zero pre-training on Hollywood screenplay syntax. Gemini has absorbed millions of scripts, enabling zero-shot formatting adherence.

Infrastructure Complexity & GPU Requirements

Self-hosting IndicTrans2 (1B–3B parameters) requires dedicated NVIDIA GPUs (VRAM, CUDA drivers, PyTorch, vLLM servers) — significant operational costs and maintenance. The Gemini API is serverless, managed, high-throughput with zero local GPU dependencies and built-in model fallback cascade.

SECTION 14

📊 Model Comparison Matrix

CAPABILITY	MURIL (GOOGLE)	AI4BHARAT	GOOGLE GEMINI ✓
Model Family	Encoder (BERT)	Encoder / Seq2Seq	Decoder Autoregressive LLM
Generative Story Writing	❌ Impossible	❌ Incapable	**✓ Hollywood format**
Max Context Window	512 tokens	1,024–2,048 tokens	**1,000,000+ tokens**
Tenglish/Hinglish Code-Switching	❌ Native script only	❌ Formal translation	**✓ Natural romanized slang**
12-Dimension Script Doctoring	❌ No reasoning	❌ No reasoning	**✓ Advanced dramatic analysis**
Studio Coverage & Scoring	❌ No	❌ No	**✓ Full rubric analysis**
Hollywood Screenplay Formatting	❌ Unknown syntax	❌ Unknown syntax	**✓ Zero-shot adherence**
Infrastructure	Dedicated GPU required	Dedicated GPU required	**✓ Serverless managed API**

SUMMARY

MuRIL and AI4Bharat are exceptional for formal Indian-language classification and document translation. However, for a generative cinematic screenplay studio requiring long-context reasoning, strict screenplay formatting, deep dramatic critique, and natural romanized code-switching, modern generative frontier LLMs (Gemini) are the technically superior and architecturally appropriate choice.